

优先队列

普通的队列是一种先进先出的数据结构，元素在队列尾追加，而从队列头删除。在优先队列中，元素被赋予优先级。当访问元素时，具有最高优先级的元素最先删除。优先队列具有最高级先出的行为特征。

也可以理解为，当把一个元素插入优先队列的时候，他会按照制定的优先顺序把该元素按照优先级插入对应的位置。但是这种寻找合适位置的时间复杂度为 $O(\log n)$ 而不是 $O(n)$ 。用到的是一种叫堆的数据结构。

STL 优先队列

头文件: `#include<queue>`

创建: `priority_queue<int> q;`

入队(push): `q.push(x);`

出队(pop): `q.pop();`

队列为空: `q.empty();`

队列的长度: `q.size();`

取队首元素: `q.top();`

如果是使用stl库默认的优先队列，他是按照插入数的大小来确定优先级的，数字越大，优先级最高。数字越小，优先级越低。

优先队列

```
priority_queue<int> q;  
  
for(int i=1;i<=n;i++)  
{  
    scanf("%d",&x);  
    q.push(x);  
}  
while(!q.empty())  
{  
    printf("%d ", q.top());  
    q.pop();  
}
```

无论我们输入的序列是什么样子的，它出队的顺序都是按照序列的值从大到小出队的。

比如输入

1 2 3 4 5

5 4 3 2 1

2 3 1 5 4

输出顺序都是5 4 3 2 1

优先队列

如果要按照从小到大排序，则需要重载小于运算符。

```
struct node
{
    int x;
    bool friend operator < (node a, node b)
    {
        return a.x > b.x;
    }
};
priority_queue<node> q;
scanf("%d", &x);
node t;
t.x = x;
q.push(t);
```

默认的优先队列是按照从大到小排列的，所以我们需要把小于运算符重载，在函数中，默认是 $a.x < b.x$ ；我们把它重载为 $a.x > b.x$ 。也就是把大小顺序交换。

优先队列

```
struct node  
{
```

```
    int x;
```

```
    friend bool operator (a, b)
```

```
{
```

```
    return a.x > b.x;
```

```
}
```

```
};
```

```
priority_queue<node> q;
```

建议写法

优先队列

从大到小输出(默认):

```
priority_queue<int , vector<int>, less<int> > q;
```

//less表示从大到小

从小到大输出:

```
priority_queue<int , vector<int>, greater<int> > q;
```

//greater表示从小到大

最后面两个相邻的>必须要用空格隔开，不然就是表示>>，程序会报错

仅供参考

P1886 滑动窗口

现在有一堆数字共N个数字 ($N \leq 10^6$)，以及一个大小为k的窗口。现在这个从左边开始向右滑动，每次滑动一个单位，求出每次滑动后窗口中的最小值和最大值。

输入：

8 3

1 3 -1 -3 5 3 6 7

输出：

-1 -3 -3 -3 3 3

3 3 5 5 6 7

Window position	Minimum value	Maximum value
[1 3 -1] -3 5 3 6 7	-1	3
1 [3 -1 -3] 5 3 6 7	-3	3
1 3 [-1 -3 5] 3 6 7	-3	5
1 3 -1 [-3 5 3] 6 7	-3	5
1 3 -1 -3 [5 3 6] 7	3	6
1 3 -1 -3 5 [3 6 7]	3	7

$N \leq 100000, K \leq 100000;$

滑动窗口

如果直接解题的话，时间复杂度是 $O(N*K)$ ，肯定会超时。我们在做的过程中，会发现浪费了很多时间，比如，求出了前一个区间的最大最小值之后，再求接下来的区间的最大最小值，再算一遍浪费了很多时间。但是如果不算，直接比较新增加的数和出窗口的数和重复部分的最大值，会导致错误。

比如 $n=4, k=3$;

5 4 2 3

第一次求出最大值是5，求移动一次之后的最大值，因为我们不知道重叠部分的最大值是多少，所以又要重新算一遍。这道题我们可以用单调队列来优化。

滑动窗口

8 3

1 3 -1 -3 5 3 6 7

先求最大值：

- (1) $a_1=1$ ，队列为空，入队
- (2) $a_2=3$, $1<3$, 说明只要3和1处在同一个窗口下，1就肯定不是最大值. 所以就让1出队，3入队。队列中保留当前可能的最大值。
- (3) $a_3=-1$ ，虽然 $-1<3$ ，但是-1在3的后面，如果-1后面的数都小于-1，-1也有可能是最大值。所以-1入队。
- (4) -3入队
- (5) 3, -1, -3都出队，5入队。
- (6) 3入队
- (7) 5 3 出队，6入队
- (8) 6出队，7入队。

但是我们要求的是窗口内的最大最小值，而不是总的最大最小值，所以我们需要考虑窗口的大小。

滑动窗口

8 3

1 3 -1 -3 -5 3 6 7

(1) $a_1=1$. 此时队列为空。因为暂时不知道后面的数是多少，所以1可能是最大值。入队。 $head=1, tail=2$ ；此时 $i - q[head].num < k(3)$ 。 $i - q[head].num < k(3)$ 是用来判断第 i 个数和当前的队首元素是否可以在一个窗口内。



滑动窗口

8 3

1 3 -1 -3 -5 3 6 7

(2) $a_2=3$. $3>1$. 有两种情况，第一种1和3如果在一个窗口下，说明1肯定不是最大值，3可能是最大值。这种情况就出队。但是1和3可能不在一个窗口下（本题是一定在），比如-2, -1, 1, 3. 当 $i=3$ 的时候最大值是1，但是 $i \geq k$ ，我们就可以直接把队首元素输出了。因为队首元素一定是队列中最大的一个。已经输出了就不用管了，所以还是1出队，3入队。



1 3 -1 -3 -5 3 6 7

注意此时 $i=3$, 我们就要输出答案了, 输出哪一个呢? 肯定是该窗口内(队列中)最大的一个——队首元素3

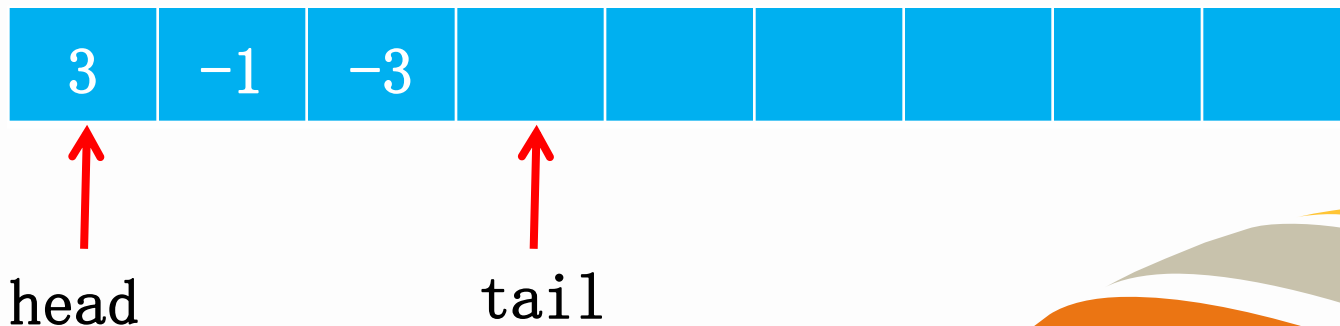


滑动窗口

8 3

1 3 -1 -3 -5 3 6 7

(4) $a_4 = -3$. 和前面-1情况一样。此时 $i - q[\text{head}].\text{num} = 2 < k$ (3), 表示当 $i=4$ 的时候, 队首元素3还在窗口内。当 $i \geq k$ 的时候, 每移动一次就要输出一个该窗口的最大值了。最大值就在队首(队列逐渐递减)。输出3. 表示 $(i-k, i)$ 的最大值是3

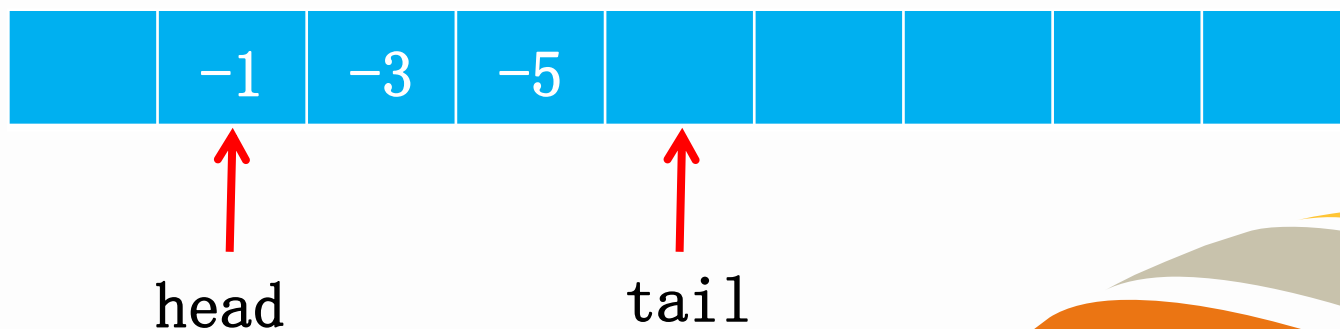


滑动窗口

8 3

1 3 -1 -3 -5 3 6 7

(5) $a_5 = -5$. 和前面一样, 入队列。但是此时 $i - q[\text{head}].\text{num} == k$, 说明当 $i = 5$ 的时候, 该队首元素不在队列中了 ($5 - 2$ 表示中间有 4 个数), 所以队首元素要先出栈了。即当前队首元素的值和第 i 个数所在的区间没有任何关系。仍然输出队首元素 -1. 表示该窗口里面的最大值是 -1.

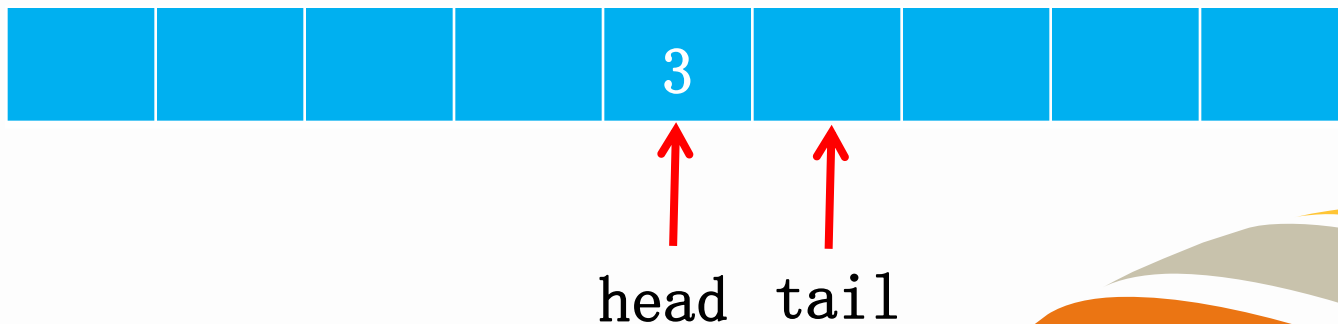


滑动窗口

8 3

1 3 -1 -3 -5 3 6 7

(6) $a_6=3$. 说明当 $i=6$ 时, 前面都不可能是最大值, 所以前面都出队。3 入队。此时 $i-q[\text{head}].\text{num}=0<3$. 输出此时的队首元素表示最大值。



滑动窗口

8 3

1 3 -1 -3 -5 3 6 7

(7) $a_7=6$. 3出队, 6入队。输出队首元素6.



head



tail

滑动窗口

8 3

1 3 -1 -3 -5 3 6 7

(8) a8=7. 6出队，7入队。输出队首元素7.

所以我们总的输出的数是3 3 -1 3 6 7。



↑ ↑
head tail

解题思路

1、单调队列具有单调性，单调队列中出队的一般是队首，所以我们让队首是最大值，这样每次直接输出队首元素就是我们要的最大值。

2、队列的含义就是表示一个窗口，队首就是表示窗口中的最大值。每次窗口向后面移动。如果当前队首元素(最大值)不在窗口中了，就出队，该窗口的最大值就是新的队首元素。如果当前队首元素仍然在窗口中，就说明该窗口的最大值仍然是该值。

3、该题的队列要始终保持队列长度 $\leq k$ 。

同样的，求最小值也是相同的方法。

滑动窗口

```
for(int i=1;i<=n;i++)  
{
```

```
    while(head<tail&&q[tail-1].x<=num[i])//出队  
    {
```

```
        tail--;
```

```
    }//如果不满足上面条件就不会有出队操作
```

```
    q[tail].x=num[i];
```

```
    q[tail].y=i;
```

```
    tail++;//当前元素入队
```

```
    while(q[head].y<=i-k)//当移动到i的位置的时候，  
    当前队首元素如果不可能和i同一个窗口。就出队
```

```
    {
```

```
        head++;
```

```
    }
```

```
    if(i>=k)//输出最大值
```

```
    printf("%d ",q[head].x);
```

```
}
```

这两段代码不能反过来，如果a1是最大值，到a3的时候就会输出，到a4的时候a1才会出队。

最好不要直接把第一个让进队列，可能出现k=1的情况



练习题

P1440: 求m区间内的最小值

P2032: 扫描

P1361: 序列合并

